VIETNAM GENERAL CONFEDERATION OF LABOR

**TON DUC THANG UNIVERSITY**

**FACULTY OF INFORMATION TECHNOLOGY**

**Tạ Chấn Nam - 52500174**

# MIDTERM ESSAY

# APPLIED CALCULUS FOR IT

**HO CHI MINH CITY, 2026**

VIETNAM GENERAL CONFEDERATION OF LABOR

**TON DUC THANG UNIVERSITY**

**FACULTY OF INFORMATION TECHNOLOGY**

**Tạ Chấn Nam - 52500174**

# MIDTERM ESSAY

# APPLIED CALCULUS FOR IT

Advised by

**Mr. Nguyen Huu An**

**HO CHI MINH CITY, 2026**

# ACKNOWLEDGEMENT

I would like to express our sincere gratitude to BSc. Nguyen Huu An, our instructor and mentor, for his valuable guidance and support throughout the mid-term report of our report about solving linear algebra logical question with relative knowledge. He is very helpful and patient in providing us with constructive feedback and suggestions to improve our work. Thanks to his friendly and harmonious personality, All of linear algebra lessons are very attractive and funny. I have learned a lot precious knowledge from his expertise, also experience in logical thinking, developed mindset and deeply understand about core value of study. I am very honored and privileged to have him as our teacher and supervisor.

*Ho Chi Minh city, 22nd December 2026.*

*Author*

*(Signature and full name)*

**Nam**

Tạ Chấn Nam

# DECLARATION OF AUTHORSHIP

I hereby declare that this is our own report and is guided by BSc. Nguyen Huu An; The content research and results contained herein are central and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the main author from different sources, which are clearly stated in the reference section.

In addition, the report also uses some comments, assessments as well as data of other authors, other organizations with citations and annotated sources.

**If something wrong happens, I'll take full responsibility for the content of my report.** Ton Duc Thang University is not related to the infringing rights, the copyrights that I give during the implementation process (if any).

*Ho Chi Minh city, 22nd December 2026*

*Author*

*(Signature and full name)*

***Nam***

Tạ Chấn Nam

# ABSTRACT

The goal of this report is represent and explain method depend on fundamental knowledge about function, graph and programming to solve mathathetic questions through Python. The report include 2 Chapter:

**Chapter 1**: Methodology of Solving Tasks.

Implement the following requirements:

Write a detailed description of the solving methods used in each subtask of the "Programming part" by:

- Providing a short description of main programming steps in the subtask
- Explaining in-detail each programming statement of the source code for the subtask

**Task** . Given two functions as follows:

$$f(x) = x^2 - 2Ax - A^2$$

$$g(x) = -x^3 + 4Ax^2 + x + A^3$$

In regard to the value of the constant A, you manually select any integer number from -99 to 80,

**Subtask a** : Find and print out intersection points of f(x) and g(x). Plot the graphs of f(x) and g(x) functions, and the intersection points in the same figure. The following information must be

**Subtask b**: Find and print out the equation of the tangent line (T) to f(x) at the point $(0, -A^2)$. Shift the graph of the function f(x) down $4A^2$ units, then find and print out intersection points of the shifted graph of f(x) and the tangent line T. Plot the original graph f(x), the shifted graph of f(x), the tangent line, the tangential point, and the intersection

points in the same figure. The following information must be presented in the figure: figure title, labels of the graphs. presented in the figure: figure title, labels of the graphs.

**Subtask c**: Find and print out the equations of the tangent lines to f(x) through the point $(0, -4A^2)$. Plot the graph of f(x) function, the tangent lines, the tangential points, and the point $(0, -4A^2)$ in the same figure. The following information must be presented in the figure: figuretitle, labels of the graphs.

**Chapter 2**: Results.

Execute the source code of the "Programming part" 3 times with different values of the constant A (difference between any two values is at least 30), then create images of all inputs and corresponding outputs for each trial, and finally insert all images into this chapter. The images must be clear, and properly laid out. The images captions and descriptions are also required.

# TABLE OF CONTENT

**LIST OF FIGURES** ..... **6**
**LIST OF TABLE** ..... **7**
**CHAPTER 1. Methodology and Solving Task:** ..... **8**
1.1. *Task 1:* ..... 8
1.2. Subtask a: ..... 12
1.3. Subtask b: ..... 15
1.4. Subtask c. ..... 18
**CHAPTER 2. Result:** ..... **21**
2.1. First Exacution (A = -99). ..... 21
*2.2. Second Exacution (A = -1).* ..... 22
*2.3. Third Exacution (A = 35)* ..... 24

# LIST OF FIGURES

Figure 2.1.1: Subtask a result (A = -99) ..... 21
Figure 2.1.2: Subtask b Result (A = -99) ..... 21
Figure 2.1.3: Subtask c result (A = -99) ..... 22
Figure 2.2.1: Subtask a Result (A = 1) ..... 22
Figure 2.2.2: Subtask b Result (A = 1) ..... 23
Figure 2.2.3: Subtask c Result (A = 1) ..... 23
Figure 2.3.1: Subtask a Result (A = 35) ..... 24
Figure 2.3.2: Subtask b Result (A = 35) ..... 24
Figure 2.3.3: Subtask c Result (A = 35) ..... 25

# LIST OF TABLE

table 1.1.1 : Function and Attributes collection table. ..... 8

# CHAPTER 1. Methodology and Solving Task:

## 1.1. Task 1:

- ***Library and Function Collection table:***

table 1.1.1: Function and Attributes collection table.

| Module | Functions / Attributes | Description |
|---|---|---|
| Python Built - in Functions, Attributes | def | Define a function. |
| | while True | Create an infinite loop. |
| | try...except | Handle errors / exceptions. |
| | if...else | Conditional statements. |
| | int() | Convert to an integer. |
| | Float() | Convert to a float. |
| | print(f"...") | Print a formatted string. |
| | input() | Get user input. |
| | zip() | Combine multiple lists/iterables. |
| | min() / max() | Find minimum / maximum value. |
| | import | Load external libraries. |
| | return | Return a value from a function. |
| | .append | Add an item to a list. |
| | .is_real | Check if a value is a real number. |
| | break | Exit a loop immediately. |
| | enumerate() | Get index and value in a loop. |
| SymPy | symbols() | Define symbolic variables. |

| | | |
|---|---|---|
| Library | diff() | Calculate the derivative. |
| | subs() | Substitute a value into an expression. |
| | Eq() | Create an equation (LHS = RHS). |
| | solve() | Solve an equation. |
| | lambdify() | Convert SymPy expression to NumPy function. |
| | evalf() | Evaluate to a floating-point number. |
| | is_real | Check for real roots/numbers. |
| Matplotlib Library | plot() | Draw lines or markers. |
| | annotate() | Add text/labels to a specific point. |
| | title() | Set the graph title. |
| | xlabel() / ylabel() | Label the X / Y axes. |
| | legend() | Show the graph legend. |
| | grid(True) | Show grid lines. |
| | axhline() / axvline() | Draw horizontal / vertical lines. |
| | ylim() | Set Y-axis limits (crop view). |
| | show() | Display the graph. |
| Define Function | tangent_line() | find and print tangent line equation by using functions |
| | find_intersection() | find intersection between two equations |
| | X_range() | set the range for horizontal axis(x-axis) in range which include the intersection points and add a padding to provide a comprehensive view of |

| | | |
|---|---|---|
| | | the graph. |
| | Plot_graph () | using functions in matpylot library to figure a basic features of a graph as titile, grid by, legend, horizontal axis, vertical axis and set a limit for horizontal axis in range found from X_range function by using |

- ***General Requirements:***

Import Libraries Numpy, Sympy, Matpylotlib as np, sp and plt.

Define functions will be used in Task by def function include:

1. ***tangent_line (fx, x, x0)*** : depending on the slope-point formula

$$y = f'(x) \text{ X } (x - x_0) + f(x)$$

   to find and print tangent line equation by using functions in sympy library as sp.diff() to find derivative, .subs() to substitute symbol by a constant value.

2. ***find_intersection (equation 1, equation 2, symbol)***: find intersection between two equations by using functions in sympy as sp.eq(equation 1, equation 2) to formulate a symbolic equation, sp.solve(equation, symbols) to solve and find symbol value in equation and attributes as .is_real to filtering real roots in set of roots.

3. ***X_range(intersection_point_x)***: set the range for horizontal axis(x-axis) in range which include the intersection points and add a padding to provide a comprehensive view of the graph.

4. ***Plot_graph(titile, set of values x, equation)***: using functions in matpylot library to figure a basic features of a graph as titile by

***plt.title(), grid by plt.grid(), legend by plt.legend()*** horizontal axis by ***plt.axhline(),*** vertical axis by ***plt.axvline()*** and set a limit for horizontal axis in range found from X_range function by using ***plt.ylim(min, max)***.

Input a integer A in range from -99 to 80: Using while True structure asociate with ***Try - Except*** structure to build a infinite loop and break when meet integer number. Inside the loop, using condition ***if - else*** to Enter A again if A is out of the set range.

Define symbol x by function ***sp.symbols().***

Define the expressions for f(x), g(x), shifted f(x)(fxdown4) and point A1 for subtask 1 and point A2 for subtask c based on the provided formulas.

- ***Source code:***

```
iimport sympy as sp
import numpy as np
import matplotlib.pyplot as plt

# 1. Function
def tagent_line(fx, x, a):
    fx_diff = sp.diff(fx, x)
    y = fx.subs(x, a)
    line = (x - a) * fx_diff.subs(x, a) + y
    return line

def find_intersection(eq1, eq2, x):
    Eq = sp.Eq(eq1, eq2)
    roots = sp.solve(Eq, x)
    real_x = [float(r.evalf()) for r in roots if r.is_real]
    real_y = [float(eq2.subs(x, rx).evalf()) for rx in real_x]
    return real_x, real_y

def X_range(r_vals, A):
    important = r_vals + [A]
    x_max = max(important) + 15
```

```
    x_min = min(important) - 15
    x_vals2 = np.linspace(x_min, x_max, 400)
    return x_vals2

def plot_graphs(graph_title, x_vals, fx_num):
    plt.title(graph_title)
    plt.legend()
    plt.grid(True)
    y_f_vals = fx_num(x_vals)
    y_min_fx = min(y_f_vals)
    y_max_fx = max(y_f_vals)
    plt.ylim(y_min_fx - 10, y_max_fx + 10)
    plt.axhline(0, color='black', linewidth=0.8)
    plt.axvline(0, color='black', linewidth=0.8)
    plt.show()
# Task 1
# Input
x, x0 = sp.symbols('x x0')
while True:
    try:
        A = int(input("Enter integer A (-99 -> 80):  "))
        if -99 <= A <= 80:
            break
        print("Integer A must range in [-99, 80]. Please enter again!")
    except ValueError:
        print("Please enter an integer!")

# Declare expresstion

stt = 1
fx = x**2 - 2*A*x - A**2
gx = -x**3 + 4*A*x**2 + x + A**3
```

## *1.2. Subtask a:*

- ***Issue:***

Nature of the Equation: Since f(x) is a quadratic function and g(x) is a cubic function, the equation f(x) = g(x) results in a cubic equation. Solving a cubic equation can yield multiple roots, including both real

and complex numbers. Therefore, we must filter for only real roots to identify the physical intersection points.

Visualization Range: When plotting, focusing only on the immediate vicinity of the intersection points may obscure the overall behavior of the functions. To provide a comprehensive view, we need to extend the x-axis range significantly beyond the roots.

- ***Methodology:***

Identify Intersections: Solve the equation $f(x) = g(x)$ to find the points where the two functions meet.

Calculate Coordinates: Once the real roots are found, determine the corresponding y-values by substituting the x-values back into either function f(x).

Filter and Output: Extract the real roots and display the coordinates of the intersection points.

Visualization: plot f(x) and g(x), highlighting the intersection points.

- ***Source Code:***

```
# Subtask a
#Step 1: Find the intersection points between fx and gx
rx_vals, ry_vals = find_intersection(fx, gx, x)
for i, (rx, ry) in enumerate(zip(rx_vals , ry_vals), 1):
    print(f"1a. Intersection point {i} : ({rx:.2f} , {ry:.2f} )")

# Step 2: Graphing
fx_num = sp.lambdify(x, fx, 'numpy')
gx_num = sp.lambdify(x, gx, 'numpy')

#Define X range
x_vals = X_range(rx_vals, A)
plt.plot(x_vals, fx_num(x_vals), label='f(x)', color='blue')
plt.plot(x_vals, gx_num(x_vals), label='g(x)', color='red')
```

```
# Plot intersection point
for rx, ry in zip(rx_vals, ry_vals):
    plt.plot(rx, ry, 'ko', label=f'Intersection point {stt}')
    plt.annotate(f'I{stt}({rx:.1f}, {ry:.1f})', (rx, ry))
plot_graphs("Intersection of f(x) and g(x)", x_vals, fx_num)
```

- ***Detail Step:***

**Step 1: Find the intersection point Filtering Real Roots**

Using function ***find_intersection()*** to find intersection point between f(x) and g(x) and ilter out complex numbers and obtain decimal approximations of the real roots return a list of real vertical roots.

**Step 2: Data Visualization with Matplotlib**

Function Conversion: transform SymPy expressions (f(x), g(x)) into numerical functions compatible with NumPy by using ***sp.lambdify()***.

Define horizontal range: using function ***np.linspace()*** in range intersection back and forward a padding point horizontal coordinate.

Plotting: Use plt.plot to draw the curves for f(x) and g(x) in X-range.

Mark Intersections: Using the loop structure for-in and zip() with Vertical real roots list and Horizon value list. Then inside the loop, with each vertical and horizon value, using function plt.plot() to highlight intersection point.

Figure the graph: using function ***plot_graph()*** to name and show a completed graph.

## *1.3. Subtask b:*

- ***Issue:***

Find tagent line equation and intersection point between tagent line and f(x) after converted.

Visualization Range: When plotting, focusing only on the immediate vicinity of the intersection points may obscure the overall behavior of the functions. To provide a comprehensive view, we need to extend the x-axis range significantly beyond the roots.

- ***Methodology:***

Tangent Line Derivation: Find the equation of the tangent line to f(x) at point $A_1(0, -A^2)$.

Intersection Analysis: Determine the intersection points between the shifted function $f(x) - 4A^2$ and the tangent line at A.

Coordinate Calculation: Filter the real roots and calculate the corresponding y-values for these points.

Visualization: Plot the function $f(x) - 4A^2$ and the tangent line, highlighting both the intersection points and point A on the graph.

- ***Source code:***

```
# Subtask b
# Step 1: find shifted f(x) and define shifted f(x) and A1(0, -A2)
fxdown4 = fx - 4*A**2
A_pointx = 0
A_pointy = -A**2

# Step 2: Find the tangent line at point A
# Using the same function tagent_line to find the tangent line at A
tagent_line_at_A = tagent_line(fx, x, A_pointx)
```

```
print(f"\nTangent line equation of f(x) at point A({A_pointx}, {A_pointy}): y =
{tagent_line_at_A}")

# Step 3: Find the intersection points between fx - 4A^2 and tangent line at point A
rx_vals2, ry_vals2 = find_intersection(tagent_line_at_A, fxdown4, x)
print("The intersection points between fx - 4A^2 and tangent line at point A1:")
for i, (rx1, ry1) in enumerate(zip(rx_vals2, ry_vals2), 1):
    print(f"Intersection point {i} : ({rx1:.2f} , {ry1:.2f} )")

# Step 4: Graphing
# Convert  sympy expressions to numpy functions for plotting
fxdown4_num = sp.lambdify(x, fxdown4, 'numpy')
tagent_num = sp.lambdify(x, tagent_line_at_A, 'numpy')

x_vals2 = X_range(rx_vals2, A)
plt.plot(x_vals2, tagent_num(x_vals2), label='Tangent Line (T)', color='orange', linewidth=2)
plt.plot(x_vals2, fx_num(x_vals2), label='Original f(x)', color='blue')
plt.plot(x_vals2, fxdown4_num(x_vals2), label='Shifted fx - 4A^2', color='green',
linewidth=2)

# plot intersection points
for rx1, ry1 in zip(rx_vals2, ry_vals2):
    plt.plot(rx1, ry1, 'ko', label = f"intersection points {stt}")
    plt.annotate(f'I{stt}({rx1:.1f}, {ry1:.1f})', (rx1, ry1), textcoords="offset points", xytext=(8,
0))
    stt+=1

# plot point A
plt.plot(A_pointx, A_pointy, 'ro', label = 'point A')
plt.annotate(f' A({A_pointx}, {A_pointy})', (A_pointx, A_pointy), textcoords="offset
points", xytext=(-20, 10), color='red')
name = f"Intersection of (fx - 4A^2) and tangent line at A (A = {A})"

#show the graph
plot_graphs(name, x_vals2, fx_num)
```

- ***Detail Step:***

**Step 1: Converted f(x)**

Subtract f(x) for $4A^2$ to shift f(x) down so the equation after converted is $f(x) - 4A^2$ named fxdown4.

**Step 2: Define tagent line at A**

Using function ***tangent_line()*** to find tangent line equation at point $A_1$

**Step 3: Find the intersection point Filtering Real Roots**

Using function ***find_intersection()*** to find intersection point between shifted f(x) and tangent line equation at point $A_1$ then filter out complex numbers and obtain decimal approximations of the real roots return a list of real vertical roots.

**Step 4: Data Visualization with Matplotlib**

Function Conversion: transform SymPy expression shifted f(x) into numerical functions compatible with NumPy by using ***sp.lambdify()***.

Define horizontal range: using function ***X_range()*** with intersection point horizontal coordinate.

Plotting: using function ***plt.plot()*** to plot f(x) and g(x) in X-range.

Mark Intersections: Using the loop structure ***for-in*** and ***zip()*** with Vertical real roots list and Horizon value list. Then inside the loop, with each vertical and horizon value, using function ***plt.plot()*** and ***plt.annonate()*** to highlight intersection point.

Figure the graph: using function ***plot_graph()*** to name and show a completed graph.

## *1.4. Subtask c.*

- ***Issue:***

Point A does not lie on the graph of f(x). Therefore, we must treat A as a point in space that lies on the tangent line to subsequently find the actual point of tangency, A'.

Visualization Range: When plotting, focusing only on the immediate vicinity of the intersection points may obscure the overall behavior of the functions. To provide a comprehensive view, we need to extend the x-axis range significantly beyond the roots.

- ***Methodology:***

Determine tagential point A': using point A lines on tagent line and tagent line formula.

Tangent Line Derivation: Find the equation of the tangent line to f(x) at point A'.

Intersection Analysis: Determine the intersection points between the shifted function f(x) and the tangent line at A'.

Coordinate Calculation: Filter the real roots and calculate the corresponding y-values for these points.

Visualization: Plot the function $f(x) - 4A^2$ and the tangent line, highlighting both the intersection points and point A on the graph.

- ***Source code:***

```
#Subtask c
#Declare point A2
A2_pointx = 0
A2_pointy = -4*A**2
#step 1: Find the vertical tangential point  through point A2(0, -4A^2)
```

```
fx_diff = sp.diff(fx, x)
tagent_line_through_A2 = (A2_pointx - x0)*fx_diff.subs(x, x0) + fx.subs(x, x0)
find_x0_eq = sp.Eq(A2_pointy, tagent_line_through_A2)
solve_x0 = sp.solve(find_x0_eq, x0)
line = [fx]
tagential_points_x = []
tagential_points_y = []

#step 2: Find the tangent line equations of f(x) through point A2(0, -4A^2) and find
the tangential points
print("Tangent line equations of f(x) through point A(0, -4*A**2)")
for i, x0_value in enumerate(solve_x0, 1):
    print(f"Tangent line {i}: y{i} = {tagent_line(fx, x, x0_value)}")
    tagentpoint = sp.solve(sp.Eq(tagent_line(fx, x, x0_value), fx), x)
    tagential_pointsx,tagential_pointsy = find_intersection(tagent_line(fx, x,
x0_value), fx, x)
    tagential_points_x.append(tagential_pointsx[0])
    tagential_points_y.append(tagential_pointsy[0])
    line.append(tagent_line(fx, x, x0_value))
#Step 3
#Define xrange
x_vals3 = X_range(tagential_points_x, A)

#Plotting equation
for i in range(1, len(line)):
    tagent_num = sp.lambdify(x, line[i], 'numpy')
    plt.plot(x_vals3, tagent_num(x_vals3), label=f'Tangent Line {i}')
plt.plot(x_vals3, fx_num(x_vals3), label='f(x)', color='blue')

#plot tanegential point and A2 point
stt = 1
for rx2, ry2 in zip(tagential_points_x, tagential_points_y):
    plt.plot(rx2, ry2, 'ko', label =f'intersection points {stt}')
    plt.annotate(f' I{stt}({rx2:.1f}, {ry2:.1f})', (rx2, ry2))
    stt+=1
plt.plot(A2_pointx, A2_pointy, 'ro', label = f'Point A({A2_pointx}, {A2_pointy})')
plt.annotate(f' A({A2_pointx}, {A2_pointy})', (A2_pointx, A2_pointy))
plot_graphs(f"Tangent lines to f(x) through A(0, -4A^2) (A={A})", x_vals3,
fx_num)
```

- ***Detail Step:***

**Step 1: Find Tagential point**

Define x0 as a symbols by using ***sp.symbols()***.

Find derivative of f(x) by ***sp.diff()*** and use in slope point formula as variables then formulate with unknown variable $x_0$.

Solving function to find possible roots by ***sp.solve()*** and substitile roots into slope-point formula to infer the tagent line equation at $x_0$.

**Step 2: Find the intersection point Filtering Real Roots**

Using function find_intersection() to find intersection point between $f(x)$ and tangent line equation at point $A'(x_0, f(x_0))$ then filter out complex numbers and obtain decimal approximations of the real roots return a list of real vertical roots and find the tangential coordinate.

**Step 3: Data Visualization with Matplotlib**

Transform SymPy expressions (Tangent Line at A') into numerical functions compatible with NumPy by using ***sp.lambdify()***.

Define horizontal range: using ***function X_range()*** with intersection point horizontal coordinate maximum real root, adding a padding to provide a comprehensive view of the graph.

Plotting: Use ***plt.plot()*** to plot f(x) and tangent line equations at A' in X-range.

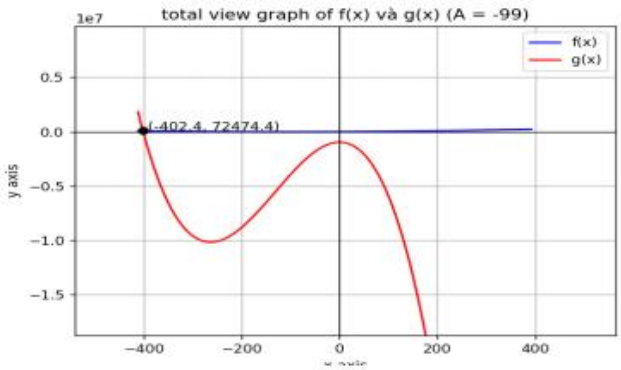
Mark Intersections: Using the loop structure ***for-in*** and ***zip()*** with Vertical real roots list and Horizon value list. Then inside the loop, with each vertical and horizon value, using function ***plt.plot()*** and ***plt.annonate()*** to highlight and mark intersection point.

Figure the graph: using function ***plot_graph()*** to name and show a completed graph.

# CHAPTER 2. Result:

## *2.1. First Exacution (A = -99).*

### - *Subtask a*

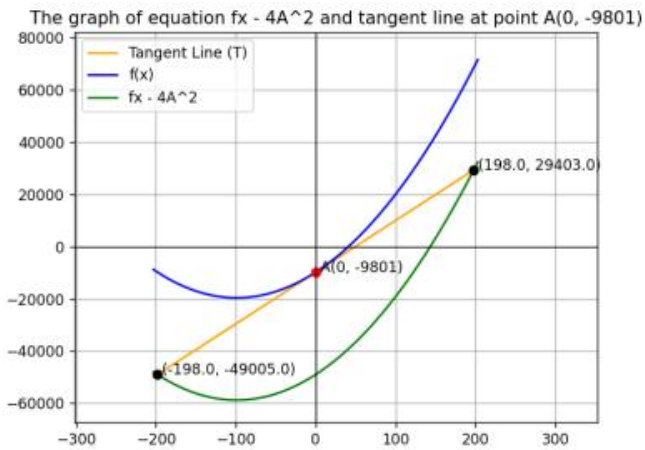



```
Enter integer A (-99 -> 80):  -99

f(x) = x**2 + 198*x - 9801
g(x) = -x**3 - 396*x**2 + x - 970299
1a. Intersection point 1 : (-402.44 , 72474.44 )
```

Figure 2.1.1: Subtask a result (A = -99)

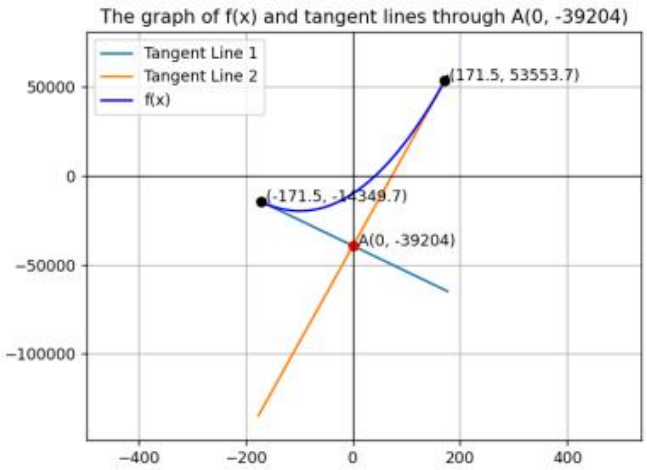
### - *Subtask b*




```
Enter integer A (-99 -> 80):  -99
tangent line equation of f(x) at point A(0, -9801): 198*x - 9801
the intersection points between fx - 4A^2 and tangent line at point A:
Intersection point 1 : (-198.00 , -49005.00 )
Intersection point 2 : (198.00 , 29403.00 )
```

Figure 2.1.2: Subtask b Result (A = -99)

- ***Subtask c***

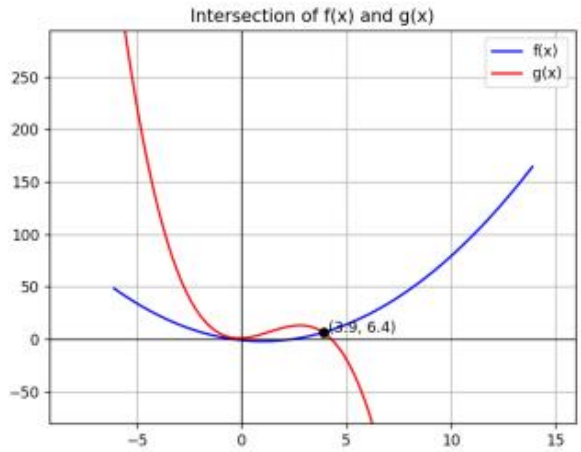



```
Enter integer A (-99 -> 80):  -99
Tangent line equations of f(x) through point A(0, -4*A**2)
Tangent line 1: y1 = (198 - 198*sqrt(3))*(x + 99*sqrt(3)) - 19602*sqrt(3) + 19602
Tangent line 2: y2 = (198 + 198*sqrt(3))*(x - 99*sqrt(3)) + 19602 + 19602*sqrt(3)
```

Figure 2.1.3: Subtask c result (A = -99)

## *2.2. Second Exacution (A = -1).*

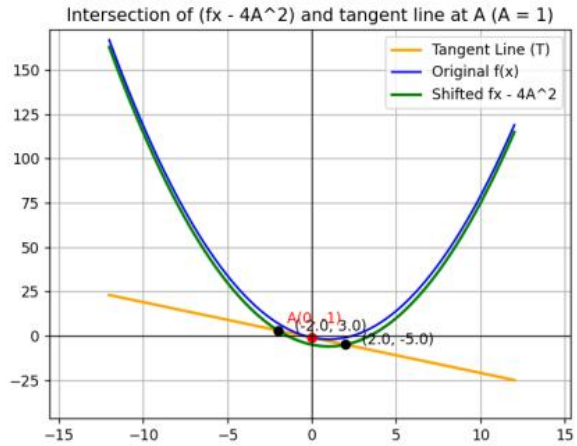
- ***Subtask a***




```
Enter integer A (-99 -> 80):  1
1a. Intersection point 1 : (3.90 , 6.41 )
```

Figure 2.2.1: Subtask a Result (A = 1)

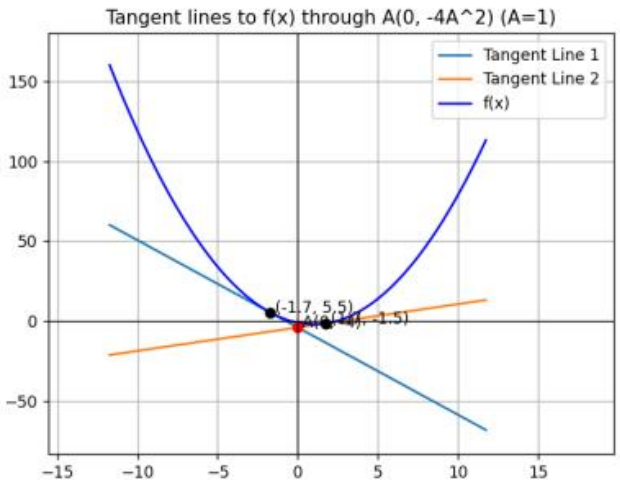
## - *Subtask b*




```
Tangent line equation of f(x) at point A(0, -1): y = -2*x - 1
The intersection points between fx - 4A^2 and tangent line at point A:
Intersection point 1 : (-2.00 , 3.00 )
Intersection point 2 : (2.00 , -5.00 )
```

Figure 2.2.2: Subtask b Result (A = 1)

## - *Subtask c*

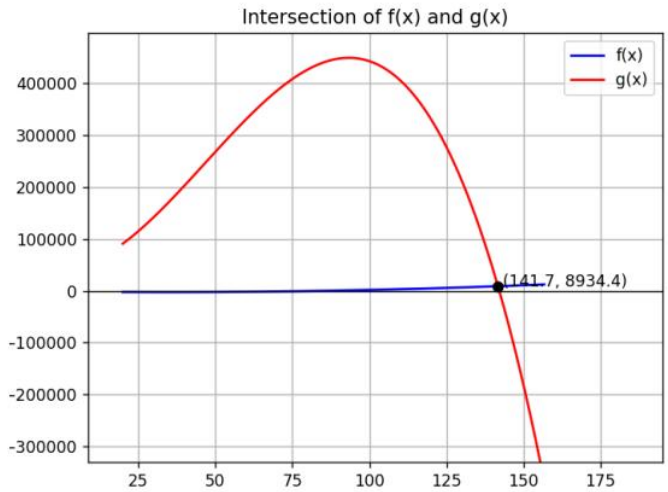



```
Tangent line equations of f(x) through point A(0, -4*A**2)
Tangent line 1: y1 = (-2*sqrt(3) - 2)*(x + sqrt(3)) + 2 + 2*sqrt(3)
Tangent line 2: y2 = (-2 + 2*sqrt(3))*(x - sqrt(3)) - 2*sqrt(3) + 2
```

Figure 2.2.3: Subtask c Result (A = 1)

## 2.3. *Third Exacution (A = 35)*

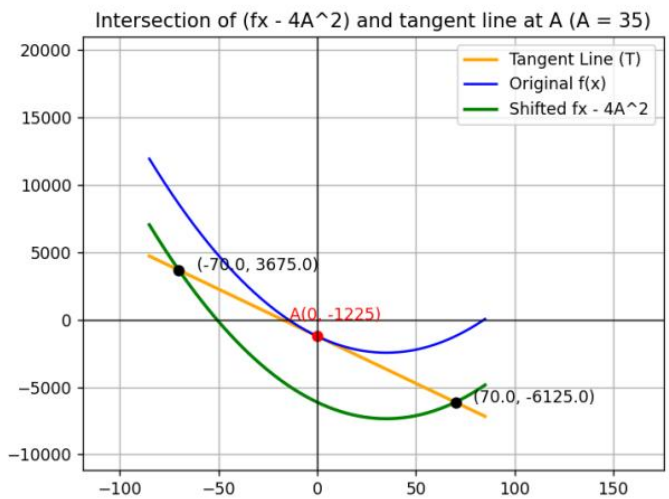
### - *Subtask a*




```
Enter integer A (-99 -> 80):  35
1a. Intersection point 1 : (141.70 , 8934.35 )
```

Figure 2.3.1: Subtask a Result (A = 35)

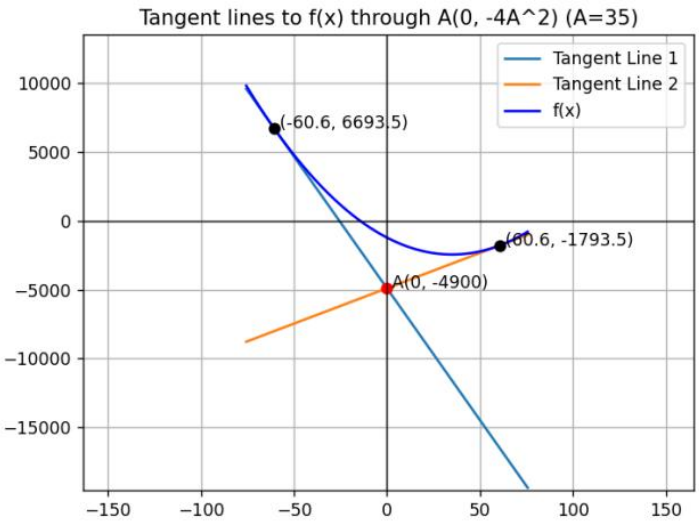
### - *Subtask b*




```
Tangent line equation of f(x) at point A(0, -1225): y = -70*x - 1225
The intersection points between fx - 4A^2 and tangent line at point A1:
Intersection point 1 : (-70.00 , 3675.00 )
Intersection point 2 : (70.00 , -6125.00 )
```

Figure 2.3.2: Subtask b Result (A = 35)

## - *Subtask c*




```
Tangent line equations of f(x) through point A(0, -4*A**2)
Tangent line 1: y1 = (-70*sqrt(3) - 70)*(x + 35*sqrt(3)) + 2450 + 2450*sqrt(3)
Tangent line 2: y2 = (-70 + 70*sqrt(3))*(x - 35*sqrt(3)) - 2450*sqrt(3) + 2450
```

Figure 2.3.3: Subtask c Result (A = 35)